

INHA UNIVERSITY TASHKENT

FALL SEMESTER 2024

SOC 3010

OPERATING SYSTEM

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR

DR. A. R. NASEER

HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

SOC 3010

OPERATING SYSTEM

WEEK 12 LECTURE 1

FILE TYPES & EXT2 MEMORY DATA STRUCTURES

FILE TYPES

☐ How Various File Types Use Disk Blocks

- The different types of files recognized by Ext2 (regular files, pipes, etc.) use data blocks in different ways.
- Some files store no data and therefore need no data blocks at all.

☐ Ext2 file types

<u>File type</u>	<u>Description</u>
0	Unknown
1	Regular file
2	Directory
3	Character device
4	Block device
5	Named pipe
6	Socket
7	Symbolic link

FILE TYPES

❑ Regular file

- Regular file needs data blocks only when it starts to have data.
- When first created, a regular file is empty and needs no data blocks; it can also be emptied by the `truncate( )` or `open( )` system calls.
- Both situations are common; for instance, when you issue a shell command that includes the string `>filename`, the shell creates an empty file or truncates an existing one.

❑ Directory

- Ext2 implements directories as a special kind of file whose data blocks store filenames together with the corresponding inode numbers.
- In particular, such data blocks contain structures of type `ext2_dir_entry_2`.

❑ Directory

- The fields of that structure are shown in Table.
- The structure has a variable length, because the last name field is a variable length array of up to EXT2_NAME_LEN characters (usually 255).
- Moreover, for reasons of efficiency, the length of a directory entry is always a multiple of 4 and, therefore, null characters (\0) are added for padding at the end of the filename, if necessary.
- The name_len field stores the actual filename length.

Table - Fields of an Ext2 directory entry

Type	Field	Description
__le32	inode	Inode number
__le16	rec_len	Directory entry length
__u8	name_len	Filename length
__u8	file_type	File type
char [EXT2_NAME_LEN]	name	Filename

255

- The file_type field stores a value that specifies the file type
- The rec_len field may be interpreted as a pointer to the next valid directory entry: it is the offset to be added to the starting address of the directory entry to get the starting address of the next valid directory entry.

FILE TYPES

❑ Example of the Ex2 Directory

- The `rec_len` field may be interpreted as a pointer to the next valid directory entry: it is the offset to be added to the starting address of the directory entry to get the starting address of the next valid directory entry.
- To delete a directory entry, it is sufficient to set its `inode` field to 0 and suitably increment the value of the `rec_len` field of the previous valid entry.
- Read the `rec_len` field of Figure carefully; you'll see that the `oldfile` entry was deleted because the `rec_len` field of `usr` is set to `12+16` (the lengths of the `usr` and `oldfile` entries).

	inode	rec_len	file_type	name_len	name
0	21	12	1	2	· \0 \0 \0
12	22	12	2	2	· · \0 \0
24	53	16	5	2	h o m e 1 \0 \0 \0
40	67	28	3	2	u s r \0
52	0	16	7	1	o l d f i l e \0
68	34	12	4	2	s b i n

LINUX DISK FILE SYSTEM

Q) Consider the Ex2 Directory entry structure as shown in the table below

address	inode	rec len	name len	file type	name
0	21			2	.
	22			2	..
	46			1	file1
	53			1	home12
	0			3	kbd1
	67			2	user1
	75			7	file2
	60			4	fd1
	0			1	Oldfile23
	34			2	sbin
	89			6	sock1

Q) Identify Regular files, Directories, character devices, block devices, named pipes, sockets and symbolic links

Q) Identify the deleted files

Q) List the file names requiring \0 padding and also specify how many \0 paddings required.

Q) Write the specific values in address, rec_len and name_len fields.

LINUX DISK FILE SYSTEM

Q) Consider the Ex2 Directory entry structure as shown in the table below

address	Inode	rec len	name len	file type	name
0	21	12	1	2	. \0\0\0
12	22	12	2	2	.. \0\0
24	46	16	5	1	file1\0\0\0
40	53	16+12=28	6	1	home12\0\0
56	0	12	4	3	kbd1
68	67	16	5	2	user1\0\0\0
84	75	16	5	7	file2\0\0\0
100	60	12+16=28	3	4	fd1\0
112	0	16	9	1	Oldfile23\0\0\0
128	34	12	4	2	sbin
140	89	16	5	6	sock1\0\0\0

Q) Identify Regular files, Directories, character devices, block devices, named pipes, sockets and symbolic links

Ans: Regular files : file1, home12, Oldfile23; Directories: ., .., user1, sbin; Character device : kbd1; block device: fd1; socket : sock1; symbolic link : file2

Q) Identify the deleted files

Ans: kbd1, Oldfile23

Q) List the file names requiring \0 padding and also specify how many \0 paddings required.

Ans: file1 – 3\0, home12 – 2\0, user1 – 3\0, file2 – 3\0, fd1 – 1\0, oldfile23- 3\0, sock1-3\0

Q) Write the specific values in address, rec_len and name_len fields.

FILE TYPES

☐ Symbolic link

- If the pathname of a symbolic link has up to 60 characters, it is stored in the `i_block` field of the inode, which consists of an array of 15 4-byte integers; no data block is therefore required.
- If the pathname is longer than 60 characters, however, a single data block is required.

☐ Device file, pipe, and socket

- No data blocks are required for these kinds of files. All the necessary information is stored in the inode.

FILE TYPES

☐ Symbolic link

- If the pathname of a symbolic link has up to 60 characters, it is stored in the `i_block` field of the inode, which consists of an array of 15 4-byte integers; no data block is therefore required.
- If the pathname is longer than 60 characters, however, a single data block is required.

☐ Device file, pipe, and socket

- No data blocks are required for these kinds of files. All the necessary information is stored in the inode.

Ext2 Memory Data Structure

- For the sake of efficiency, most information stored in the disk data structures of an Ext2 partition are copied into RAM when the filesystem is mounted, thus allowing the kernel to avoid many subsequent disk read operations.
- To get an idea of how often some data structures change, consider some fundamental operations:
- When a **new file is created**, the values of the **s_free_inodes_count** field in the Ext2 superblock and of the **bg_free_inodes_count** field in the proper group descriptor must be decreased.
- If the kernel appends some data to an existing file so that the number of data blocks allocated for it increases, the values of the **s_free_blocks_count** field in the Ext2 superblock and of the **bg_free_blocks_count** field in the group descriptor must be modified.
- Even just rewriting a portion of an existing file involves an update of the **s_wtime** field of the Ext2 superblock.
- Because all Ext2 disk data structures are stored in blocks of the Ext2 partition, the kernel uses the page cache to keep them up-to-date

Ext2 Memory Data Structure

- Table specifies, for each type of data related to Ext2 filesystems and files, the data structure used on the disk to represent its data, the data structure used by the kernel in memory, and a rule of thumb used to determine how much caching is used.
- Data that is updated very frequently is always cached; that is, the data is permanently stored in memory and included in the page cache until the corresponding Ext2 partition is unmounted.
- The kernel gets this result by keeping the page's usage counter greater than 0 at all times.

Table - VFS images of Ext2 data structures

<u>Type</u>	<u>Disk data structure</u>	<u>Memory data structure</u>	<u>Caching mode</u>
Superblock	ext2_super_block	ext2_sb_info	Always cached
Group descriptor	ext2_group_desc	ext2_group_desc	Always cached
Block bitmap	Bit array in block	Bit array in buffer	Dynamic
inode bitmap	Bit array in block	Bit array in buffer	Dynamic
inode	ext2_inode	ext2_inode_info	Dynamic
Data block	Array of bytes	VFS buffer	Dynamic
Free inode	ext2_inode	None	Never
Free block	Array of bytes	None	Never

Ext2 Memory Data Structure

❑ PAGE CACHE

- The never-cached data is not kept in any cache because it does not represent meaningful information.
- Conversely, the always-cached data is always present in RAM, thus it is never necessary to read the data from disk (periodically, however, the data must be written back to disk).
- In between these extremes lies the dynamic mode.
- In this mode, the data is kept in a cache as long as the associated object (inode, data block, or bitmap) is in use; when the file is closed or the data block is deleted, the page frame reclaiming algorithm may remove the associated data from the cache.
- It is interesting to observe that inode and block bitmaps are not kept permanently in memory; rather, they are read from disk when needed.
- Actually, many disk reads are avoided and the page cache used which keeps in memory the most recently used disk blocks.

The Ext2 inode Object

❑ Creating the Ext2 Filesystem

- There are generally two stages to creating a filesystem on a disk.
- The first step is to format it so that the disk driver can read and write blocks on it.
- **Modern hard disks come preformatted from the factory** and need not be reformatted; floppy disks may be formatted on Linux using a utility program such as superformat or fdformat.
- **The second step involves creating a filesystem, which means setting up the structures**

Creating the Ext2 Filesystem

❑ Creating the Ext2 Filesystem

- Ext2 filesystems are created by the **mke2fs** utility program; it assumes the following default options, which may be modified by the user with flags on the command line:
- Block size: 1,024 bytes (default value for a small filesystem)
- Fragment size: block size (block fragmentation is not implemented)
- Number of allocated inodes: 1 inode for each 8,192 bytes
- Percentage of reserved blocks: 5 percent

Creating the Ext2 Filesystem

❑ Creating the Ext2 Filesystem

The program **mke2fs** performs the following actions:

1. Initializes the superblock and the group descriptors
2. Optionally, checks whether the partition contains defective blocks; if so, it creates a list of defective blocks.
3. For each block group, reserves all the disk blocks needed to store the superblock, the group descriptors, the inode table, and the two bitmaps.
4. Initializes the inode bitmap and the data map bitmap of each block group to 0.
5. Initializes the inode table of each block group.
6. Creates the /root directory.
7. Creates the lost+found directory, which is used by e2fsck to link the lost and found defective blocks.
8. Updates the inode bitmap and the data block bitmap of the block group in which the two previous directories have been created.
9. Groups the defective blocks (if any) in the lost+found directory.

Creating the Ext2 Filesystem

❑ Creating the Ext2 Filesystem

- Consider how an Ext2 1.44 MB floppy disk is initialized by mke2fs with the default options.
- Once mounted, it appears to the VFS as a volume consisting of 1,440 blocks; each one is 1,024 bytes in length. (roughly there are 1474 blocks)
- To examine the disk's contents, we can execute the Unix command:
`$ dd if=/dev/fd0 bs=1k count=1440 | od -tx1 -Ax > /tmp/dump_hex`
- to get a file containing the hexadecimal dump of the floppy disk contents in the /tmp directory.
- Most information on an Ext2 filesystem could also be obtained by using the dumpe2fs and debugfs utility programs.
- By looking at that file, due to the limited capacity of the disk, a single group descriptor is sufficient.
- We also notice that the number of reserved blocks is set to 72 (5 percent of 1,440) and, according to the default option, the inode table must include 1 inode for each 8,192 bytes that is, 184 inodes stored in 23 blocks.

Creating the Ext2 Filesystem

❑ Creating the Ext2 Filesystem

Table Ext2 block allocation for a floppy disk

<u>Block</u>	<u>Content</u>
0	Boot block
1	Supeblock
2	Block containing a single block group descriptor
3	Data block bitmap
4	inode bitmap
5-27	inode table: inodes up to 10: reserved (inode 2 is the root); inode 11: lost+found; inodes 12-184: free
28	Root directory (includes ., .., and lost+found)
29	lost+found directory (includes . and ..)
30-40	Reserved blocks preallocated for lost+found directory
41-1439	Free blocks

❑ Ext2 Superblock Operations

- Many VFS superblock operations have a specific implementation in Ext2, namely **alloc_inode**, **destroy_inode**, **read_inode**, **write_inode**, **delete_inode**, **put_super**, **write_super**, **statfs**, **remount_fs**, and **clear_inode**.
- The addresses of the superblock methods are stored in the **ext2_sops** array of pointers.

Creating the Ext2 Filesystem

- ❑ **Managing Ext2 Disk Space - allocating and deallocating inodes and data blocks**
- ❑ **The storage of a file on disk differs from the view the programmer has of the file in two ways:**
 - **Blocks can be scattered around the disk (although the filesystem tries hard to keep blocks sequential to improve access time), and**
 - **Files may appear to a programmer to be bigger than they really are because a program can introduce holes into them (through the lseek() system call).**

Creating the Ext2 Filesystem

❑ Managing Ext2 Disk Space

- ❖ Two main problems must be addressed when the Ext2 filesystem manages the disk space by allocating and deallocating inodes and data blocks.
- Space management must make every effort to avoid file fragmentation the physical storage of a file in several, small pieces located in non-adjacent disk blocks.

File fragmentation increases the average time of sequential read operations on the files, because the disk heads must be frequently repositioned during the read operation.

This problem is similar to the external fragmentation of RAM

Please note that fragmenting a file across block groups is quite different from the not-yet-implemented fragmentation of blocks to store many files in one block

- **Space management must be time-efficient**; that is, the kernel should be able to quickly derive from a file offset the corresponding logical block number in the Ext2 partition.
- In doing so, the kernel should limit as much as possible the number of accesses to addressing tables stored on disk, because each such intermediate access considerably increases the average file access time.

Creating the Ext2 Filesystem

❑ Creating inodes

- The **ext2_new_inode()** function creates an Ext2 disk inode, returning the address of the corresponding inode object (or NULL, in case of failure).
- The function carefully selects the block group that contains the new inode; this is done to spread unrelated directories among different groups and, at the same time, to put files into the same group as their parent directories.
- To balance the number of regular files and directories in a block group, Ext2 introduces a "debt" parameter for every block group.
- The function acts on two parameters: the address dir of the inode object that refers to the directory into which the new inode must be inserted and a mode that indicates the type of inode being created.
- The latter argument also includes the MS_SYNCHRONOUS mount flag that requires the current process to be suspended until the inode is allocated.

Deleting inodes

❑ Deleting inodes

- The **ext2_free_inode() function** deletes a disk inode, which is identified by an inode object whose address inode is passed as the parameter.
- The kernel should invoke the function after a series of cleanup operations involving internal data structures and the data in the file itself.
- It should come after the inode object has been removed from the inode hash table, after the last hard link referring to that inode has been deleted from the proper directory and after the file is truncated to 0 length to reclaim all its data blocks

LINUX DISK FILE SYSTEM

- **Q) Consider a 32-GB Ext2 partition with a 1-KB block size. Determine the approximate number of Block Groups in the disk partition**

Given: Disk partition size = 32 GB, Block size = 1 KB

- **Block Size = 1KB = 1024 bytes = 1024x8 bits = 8192 bits**
- **Data block Bitmap size = 1 block = 1KB = 8192 bits**
- **Since each bit in the Data block bitmap corresponds to a Data block,**
Total number of Data blocks in a Block Group = 8192 blocks = 8K
- ★ **In this case, each 1-KB block bitmap describes 8K data blocks**
 - **Total amount of disk space required to store 8K data blocks**
= 8K x size of each block = 8K x 1KB = 8MB.
 - **Size of a Blockgroup = 8K blocks = 8K x 1KB = 8MB**
 - **Approximate Number of Block Groups in the given 32-GB partition**
= 32GB/8MB = 32x1024/8 = 4096 Block Groups
 - **Therefore, at most 4096 block groups are needed.**
 - **Clearly, the smaller the block size, the larger the number of block groups.**

LINUX DISK FILE SYSTEM

- Q) Consider a 32-GB Ext2 partition with a 4-KB block size. Determine the approximate number of Block Groups in the disk partition

Given: Disk partition size = 32 GB, Block size = 4 KB

- Block Size = 4KB = 4096 bytes = 4096x8 bits = 32768 bits
- Data block Bitmap size = 1 block = 4KB = 32768 bits
- Since each bit in the Data block bitmap corresponds to a Data block,
Total number of Data blocks in a Block Group = 32768 blocks = 32K
- In this case, each 4-KB block bitmap describes 32K data blocks
- Total amount of disk space required to store 32K data blocks
= 32K x size of each block = 32K x 4KB = 128 MB.
- Size of a Block Group = 32K blocks = 32K x 4KB = 128MB
- Approximate Number of Block Groups in the given 32-GB partition
= 32GB/128MB = 32x1024/128 = 256 Block Groups
- Therefore, at most 256 block groups are needed.
- Clearly, the larger the block size, the smaller the number of block groups.

LINUX DISK FILE SYSTEM

- **Q) Consider a 32-GB Ext2 partition with a 8-KB block size. Determine the approximate number of Block Groups in the disk partition**

Given: Disk partition size = 32 GB, Block size = 8 KB

- **Block Size = 8KB = 8192 bytes = 8192x8 bits = 65536 bits**
- **Data block Bitmap size = 1 block = 8KB = 65536 bits**
- **Since each bit in the Data block bitmap corresponds to a Data block,**
Total number of Data blocks in a Block Group = 65536 blocks = 64K

- **In this case, each 8-KB block bitmap describes 64K data blocks**
- **Total amount of disk space required to store 64K data blocks**
= 64K x size of each block = 64K x 8KB = 512 MB.
- **Size of a Block Group = 64K blocks = 64K x 8KB = 512MB**
- **Approximate Number of Block Groups in the given 32-GB partition**
= 32GB/512MB = 64 Block Groups
- **Therefore, at most 64 block groups are needed.**
- **Clearly, the larger the block size, the smaller the number of block groups.**

LINUX DISK FILE SYSTEM

- **Q)** Consider a 512-GB Ext2 partition with a 4-KB block size. Determine the approximate number of Block Groups in the disk partition

Given: Disk partition size = 512 GB, Block size = 4 KB

- **Block Size = 4KB = 4096 bytes = 4096x8 bits = 32768 bits**
- **Data block Bitmap size = 1 block = 4KB = 32768 bits**
- **Since each bit in the Data block bitmap corresponds to a Data block,**
Total number of Data blocks in a Block Group = 32768 blocks = 32K

- **In this case, each 4-KB block bitmap describes 32K data blocks**
- **Total amount of disk space required to store 32K data blocks**
= 32K x size of each block = 32K x 4KB = 128 MB.
- **Size of a Block Group = 32K blocks = 32K x 4KB = 128MB**
- **Approximate Number of Block Groups in the given 512-GB partition**
= 512GB/128MB = 4096 Block Groups
- **Therefore, at most 4096 block groups are needed.**
- **Clearly, the larger the block size, the smaller the number of block groups.**

LINUX DISK FILE SYSTEM

- Q) Consider a 512-GB Ext2 partition with a 8-KB block size. Determine the approximate number of Block Groups in the disk partition

Given: Disk partition size = 512 GB, Block size = 8 KB

- Block Size = 8KB = 8192 bytes = 8192x8 bits = 65536 bits
- Data block Bitmap size = 1 block = 8KB = 65536 bits
- Since each bit in the Data block bitmap corresponds to a Data block,
Total number of Data blocks in a Block Group = 65536 blocks = 64K
- In this case, each 8-KB block bitmap describes 64K data blocks
- Total amount of disk space required to store 64K data blocks
= 64K x size of each block = 64K x 8KB = 512 MB.
- Size of a Block Group = 64K blocks = 64K x 8KB = 512MB
- Approximate Number of Block Groups in the given 512-GB partition
= 512GB/512MB = 1024 Block Groups
- Therefore, at most 1024 block groups are needed.
- Clearly, the larger the block size, the smaller the number of block groups.

LINUX DISK FILE SYSTEM

Q) Given a Ext2 file system having 8192 inodes in each blockgroup with blocksize 4KB, determine the total number of blocks required by the Inode Table

- In Ext2 filesystem, Inode size =128 bytes
- Given Block size=4KB,
Number inodes per Blockgroup = 8192
- A 1KB block can contain $1\text{KB}/128 = 1024/128 = 8$ inodes
- A 4KB block can contain 32 inodes
- Hence, total number of blocks in the Inode table = Number of inodes per blockgroup/ Number of inodes per block
 $= 8192/32 = 256$ blocks

LINUX DISK FILE SYSTEM

Q) Given a Ext4 file system having 8192 inodes in each blockgroup with blocksize 4KB, determine the total number of blocks required by the Inode Table

- In Ext4 filesystem, Inode size =256 bytes

- Given Block size=4KB,

Number inodes per Blockgroup = 8192

- A 1KB block can contain $1\text{KB}/256\text{bytes} = 4$ inodes

- A 4KB block can contain 16 inodes

- Hence, total number of blocks in the Inode table = Number of inodes per blockgroup/Number of inodes per block
 $= 8192/16 = 512$ blocks

LINUX DISK FILE SYSTEM

Q) Suppose that each block group contains 4,096 inodes and that we want to know the address of inode 13,021 on disk . That is, Compute Blockgroup number and which entry of the corresponding Node table contain this inode 13,021.

In Blockgroup 0 – inode 0 to 4095,

In Blockgroup 1 – inode 4096 to 8191

In Blockgroup 2 – inode 8192 to 12287

In Blockgroup 3 – inode 12288 to 16383

In this case, the inode belongs to the Blockgroup number = $13021/4096=3$, i.e., **Block group 3** and its disk address is stored in the $(13021- 3 \times 4096 = 13021 - 12288 = 733)$ **733rd** entry of the corresponding inode table.

LINUX DISK FILE SYSTEM

Q) Suppose that each block group contains 4,096 inodes and that we want to know the address of inode 28,428 on disk .

That is, Compute Blockgroup number and which entry of the corresponding Node table contain this inode 28,428.

In this case, the inode belongs to the Blockgroup number = $28428/4096=6$, i.e., Block group 6 and

its disk address is stored in the

$(28428 - 6 \times 4096 = 28428 - 24576 = 3852)$ 3852nd entry of the corresponding inode table.

LINUX DISK FILE SYSTEM

Q) In a Ext2 file system, if disk block size = 1KB, # of inodes = 2048 inodes and Blockgroups then determine the following :

4K

- a) Size of superblock = 1 block
- b) Number of Blockgroup descriptors = 4K
- c) Size of blockgroup descriptors = $4K * 24/1KB = 96$ blocks

- d) Size of data block bitmap = 1 block
- e) Size of inode bitmap = 1 block
- f) Size of inode = 128 bytes
- g) Number of inodes = 2048
- h) Size of inode table =
= size of inode x number of inodes = $128 \times 2048 = 256K = 256$ blocks

- i) Total number of data blocks in the Block group =
= Blocksize x 8 = $1KB \times 8 = 1024 \times 8 = 8K = 8192$ blocks

- j) Number of data blocks available for storing file data
= Total no. of data blocks in Block group – (1block(super block) +
96 blocks(Blockgroup descriptors)+ 1 Block (data block bitmap) +
1block(inode bitmap) + 256blocks(Inode table)) = $(8192 - 355) = 7837$ blocks

- k) Size of one blockgroup =
= $8192 \text{ blocks} \times \text{size of each block} = 8K \times 1K = 8MB$
- l) Size of the disk partition =
= No. of Blockgroups x size of one Blockgroup
= $4K \times 8MB = 32GB$

LINUX DISK FILE SYSTEM

Q) In a Ext2 file system, if disk block size = 4KB, # of inodes = 8192 inodes and 16K Blockgroups then determine the following :

- a) Size of superblock = 1 block
- b) Number of Blockgroup descriptors = 16K
- c) Size of blockgroup descriptors = $16K \times 24 / 4K = 96$ blocks
- d) Size of data block bitmap = 1 block
- e) Size of inode bitmap = 1 block
- f) Size of inode = 128 bytes
- g) Number of inodes = 8192
- h) Size of inode table =
= size of inode x number of inodes = $128 \times 8192 = 256$ blocks
- i) Total number of data blocks in the Block group =
= Blocksize x 8 = $4096 \times 8 = 32768$ blocks
- j) Number of data blocks available for storing file data
= Total no. of data blocks in Block group – (1block(super block) +
96 blocks(Blockgroup descriptors)+ 1 Block (data block bitmap) +
1block(inode bitmap) + 256blocks(Inode table)) = $(32768 - 355) = 32413$ blocks
- k) Size of one blockgroup =
= $32768 \text{ blocks} \times \text{size of each block} = 32K \times 4K = 128MB$
- l) Size of the disk partition =
= No. of Blockgroups x size of one Blockgroup
= $16K \times 128MB = 2048GB = 2TB$

LINUX DISK FILE SYSTEM

**Q) Given a 64GB partition having Ext2 file system with disk block size = 4KB,
of inodes = 8192 inodes then determine the following :**

- a) Total number of blocks in a blockgroup = $\text{blocksize} \times 8 = 4096 \times 8 = 32768 \text{ blocks}$
- b) Size of one Blockgroup = $32768 \times 4\text{KB} = 32\text{KB} \times 4\text{KB} = 128\text{MB}$
- c) Total number of Blockgroups = $64\text{GB} / 128\text{MB} = 512$
- d) Size of superblock = 1 block
- e) Size of blockgroup descriptors = $512 \times 24 / 4\text{KB} = 3 \text{ blocks}$
- f) Size of data block bitmap = 1 block
- g) Size of inode bitmap = 1 block
- h) Size of inode = 128 bytes
- i) Number of inodes = 8192
- j) Size of inode table =
= size of inode $\times$ number of inodes = $128 \times 8192 = 1024\text{KB} / 4\text{KB} = 256 \text{ blocks}$
- k) Number of data blocks available for storing file data
= Total no. of data blocks in Block group – (1block(super block) +
3 blocks(Blockgroup descriptors)+ 1 Block (data block bitmap) +
1block(inode bitmap) + 256blocks(Inode table))
= $32768 - 262 = 32506 \text{ blocks}$

LINUX DISK FILE SYSTEM

Q) Given a 128 GB partition having Ext2 file system with disk block size = 4KB, # of inodes = 8192 inodes then determine the following :

- a) Total number of blocks in a blockgroup = $\text{blocksize} \times 8 = 4096 \times 8 = 32768 \text{ blocks}$
- b) Size of one Blockgroup = $32768 \times 4\text{KB} = 32\text{KB} \times 4\text{KB} = 128\text{MB}$
- c) Total number of Blockgroups = $128\text{GB} / 128\text{MB} = 1024$
- d) Size of superblock = 1 block
- e) Size of blockgroup descriptors = $1024 \times 24 / 4\text{K} = 6 \text{ blocks}$
- f) Size of data block bitmap = 1 block
- g) Size of inode bitmap = 1 block
- h) Size of inode = 128 bytes
- i) Number of inodes = 8192
- j) Size of inode table =
= $\text{size of inode} \times \text{number of inodes} = 128 \times 8192 = 128 \times 8\text{KB} / 4\text{KB} = 256 \text{ blocks}$
- k) Number of data blocks available for storing file data
= Total no. of data blocks in Block group – (1block(super block) + 6 blocks(Blockgroup descriptors)+ 1 Block (data block bitmap) + 1block(inode bitmap) + 256 blocks(Inode table))
= $32768 - 265 = 32503 \text{ blocks}$

SOC 3010 OPERATING SYSTEMS

Reading Assignments

Chapter 18 of Text Book

Text Book

➤ Understanding the Linux Kernel

- Daniel P/ Bovet and Marco Cesati
Edition, O' Reilly Media 2005, ISBN 978-0596005658**

DISK FILE SYSTEM

□ Questions

- * What are the different file types recognized by Ext2 disk file system?
- * Explain the following file types:
 - a)Regular file (Type 1)
 - b)Directory (Type 2)
 - c)Character device (Type 3)
 - d)Block device (Type 4)
 - e)Named pipe (Type 5)
 - f)Socket (Type 6)
 - g)Symbolic link (Type 7)
- * Indicate the types of files which require no data blocks on the disk.
- * Explain why no data block is required on the disk if symbolic link has upto 60 characters.
- * Describe the two steps used in general to create a filesystem on a disk.
- * Write the command used in linux to create a filesystem on a disk. Indicate the correct syntax of the command by giving an example.
- * Indicate which of the following types of fields in the Ext2 data structures are :

Always cached or stored in Dynamic memory or Never cached :

- | | |
|---------------------|-------|
| a) Super block | |
| b) Group Descriptor | |
| c) Block bitmap | |
| d) Inode bitmap | |
| e) Inode | |
| f) Data block | |
| g) Free inode | |
| h) Free block | |

DISK FILE SYSTEM

□ Questions

- * Consider the Ex2 Directory entry structure as shown in the table below :

address	inode	rec len	name len	file type	name
0	21			2	.
	22			2	..
	46			1	file1
	53			1	home12
	0			3	kbd1
	67			2	user1
	75			7	file2
	60			4	fd1
	0			1	Oldfile23
	34			2	sbin
	89			6	sock1

- * Identify Regular files, Directories, character devices, block devices, named pipes, sockets and symbolic links
- * Identify the deleted files
- * List the file names requiring \0 padding and also specify how many \0 paddings required.
- * Write the specific values in address, rec_len and name_len fields.

DISK FILE SYSTEM

□ Questions

- * List out all the actions performed by make2fs program.
- * State the purpose of a lost+found directory.
- * Consider an Ext2 1.44 MB floppy disk which is initialized by make2fs with the default options - Block size: 1,024 bytes, Number of allocated inodes : 1 inode for each 8,192 bytes, Percentage of reserved blocks: 5 percent.

Compute the following:

Total number of blocks in the floppy disk considering its size.

Total number of reserved blocks

Number of Block group descriptors

Number of inodes in the inode table

Size of inode table in terms of blocks

- * State the purpose of the following command :

\$ dd if=/dev/sda1 of=/dev/sda6

- * State the operation performed by the following command :

\$ dd if=/dev/fd0 bs=1k count=1440 | od -tx1 -Ax > /tmp/dump_hex

- * State the two main problems that must be addressed when the Ext2 filesystem manages the disk space by allocating and deallocating inodes and data blocks.

DISK FILE SYSTEM

□ Questions

- * State the purpose of **sparse_super** feature flag in Ext4 file system.

- * If disk block size = 1KB, # of inodes = 8192 inodes and 8K Blockgroups then determine the following
 - a) size of superblock,
 - b) size of blockgroup descriptors,
 - c) size of data block bitmap,
 - d) size of inode bitmap,
 - e) size of inode and size of inode table,
 - f) number of data blocks and size of data blocks
 - g) Size of one blockgroup
 - h) Size of the disk partition.

- * If disk block size = 2 KB, # of inodes = 8192 inodes and 16K Blockgroups then determine the following
 - a) size of superblock,
 - b) size of blockgroup descriptors,
 - c) size of data block bitmap,
 - d) size of inode bitmap,
 - e) size of inode and size of inode table,
 - f) number of data blocks and size of data blocks
 - g) Size of one blockgroup
 - h) Size of the disk partition.

SOC 3010

OPERATING SYSTEM

EXT DATA BLOCK ADDRESSING

❑ Data Blocks Addressing

- Each nonempty regular file consists of a group of data blocks .
- Such blocks may be referred to either by their relative position inside the file their file block number or by their position inside the disk partition their logical block number
- Deriving the logical block number of the corresponding data block from an offset f inside a file is a two-step process:
 1. Derive from the offset f the file block number the index of the block that contains the character at offset f .
 2. Translate the file block number to the corresponding logical block number.

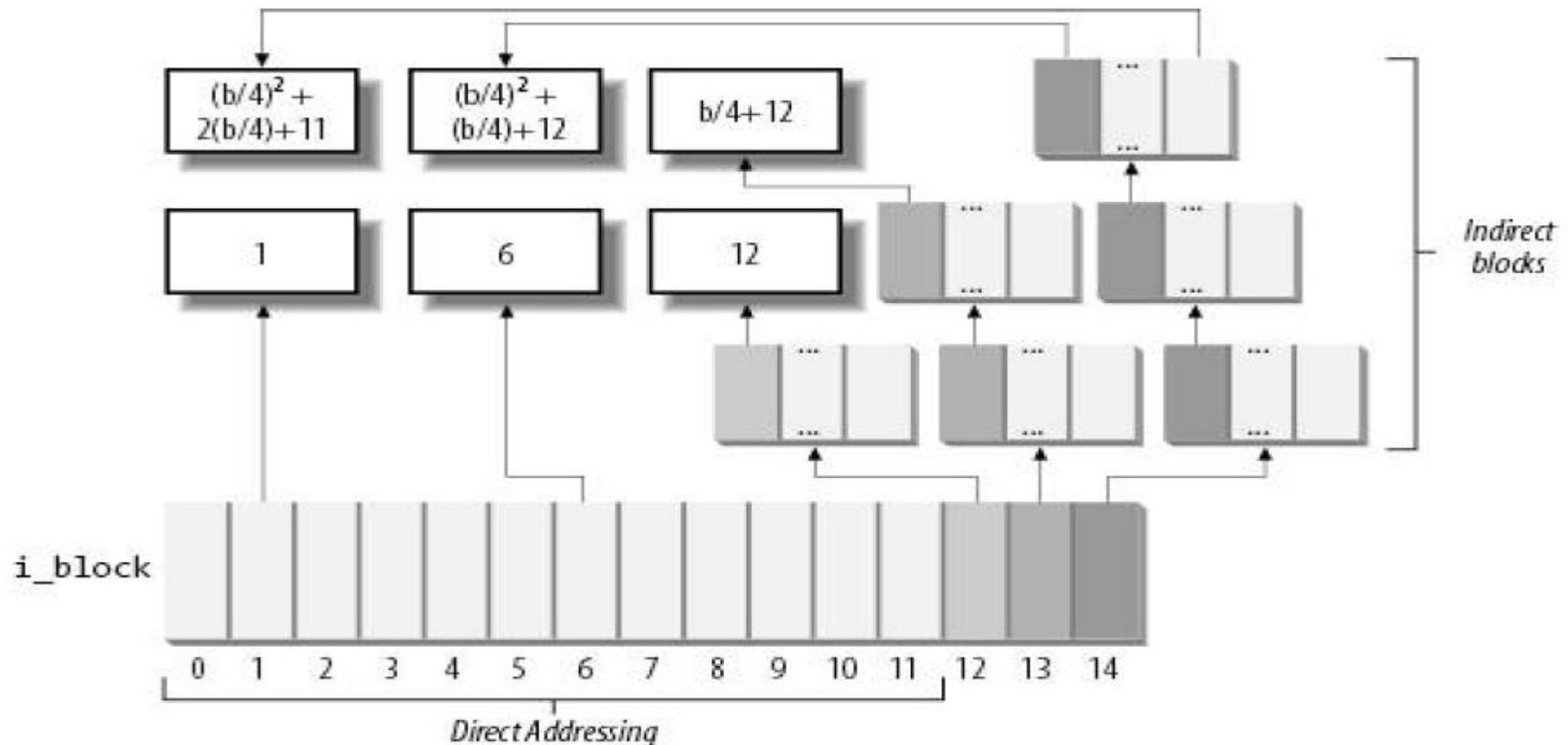
□ Data Blocks Addressing

- Because Unix files do not include any control characters, it is quite easy to derive the file block number containing the f th character of a file: simply take the quotient of f and the filesystem's block size and round down to the nearest integer.
- For instance, let's assume a block size of 4 KB.
 - If f is smaller than 4,096, the character is contained in the first data block of the file, which has file block number 0.
 - If f is equal to or greater than 4,096 and less than 8,192, the character is contained in the data block that has file block number 1, and so on.
- This is fine as far as file block numbers are concerned.
- However, translating a file block number into the corresponding logical block number is not nearly as straightforward, because the data blocks of an Ext2 file are not necessarily adjacent on disk.
- The Ext2 filesystem must therefore provide a method to store the connection between each file block number and the corresponding logical block number on disk.
- This mapping, which goes back to early versions of Unix from AT&T, is implemented partly inside the inode.
- It also involves some specialized blocks that contain extra pointers, which are an inode extension used to handle large files.

Data Blocks Addressing

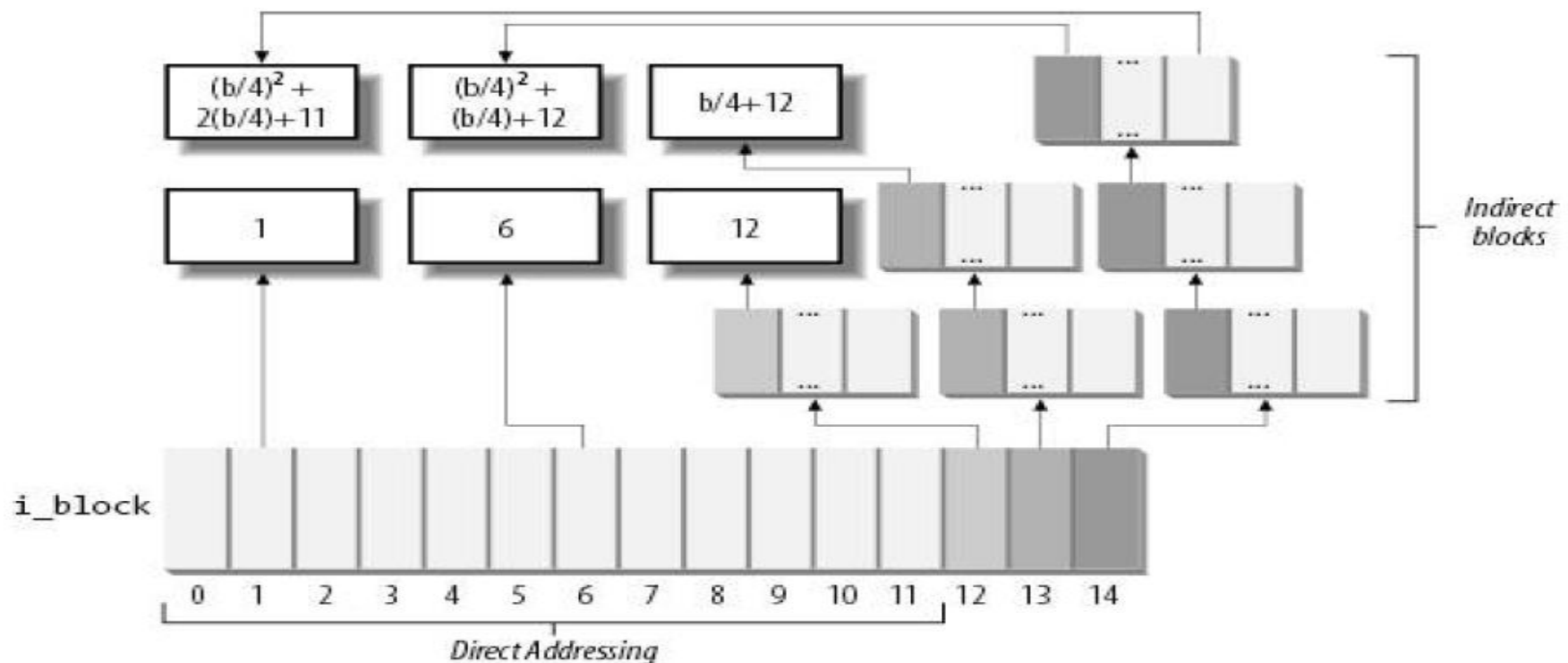
□ Data Blocks Addressing

- The `i_block` field in the disk inode is an array of `EXT2_N_BLOCKS` components that contain logical block numbers.
- Assume that `EXT2_N_BLOCKS` has the default value, namely 15.
- The array represents the initial part of a larger data structure, which is illustrated in **Figure-Data structures used to address the file's data blocks**



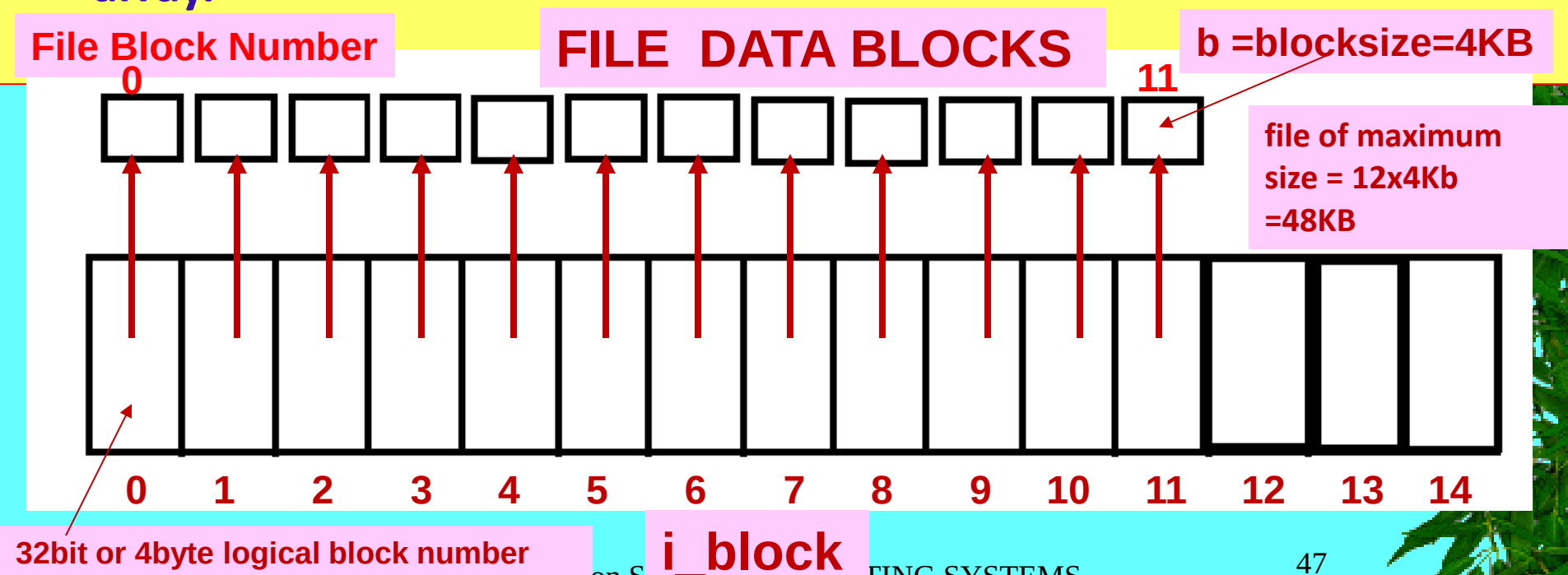
□ Data Blocks Addressing

- The `i_block` field in the disk inode is an array of `EXT2_N_BLOCKS` components that contain logical block numbers.
- Assume that `EXT2_N_BLOCKS` has the default value, namely 15.
- The array represents the initial part of a larger data structure, which is illustrated in **Figure-Data structures used to address the file's data blocks**
- As can be seen in the figure, the 15 components of the array are of 4 different types: FIRST 0 TO 11 COMPONENTS POINTING TO DATA BLOCKS, 12, 13, & 14 COMPONENTS POINTING TO INDIRECT BLOCKS



❑ Data Blocks Addressing – 0 to 11 Components

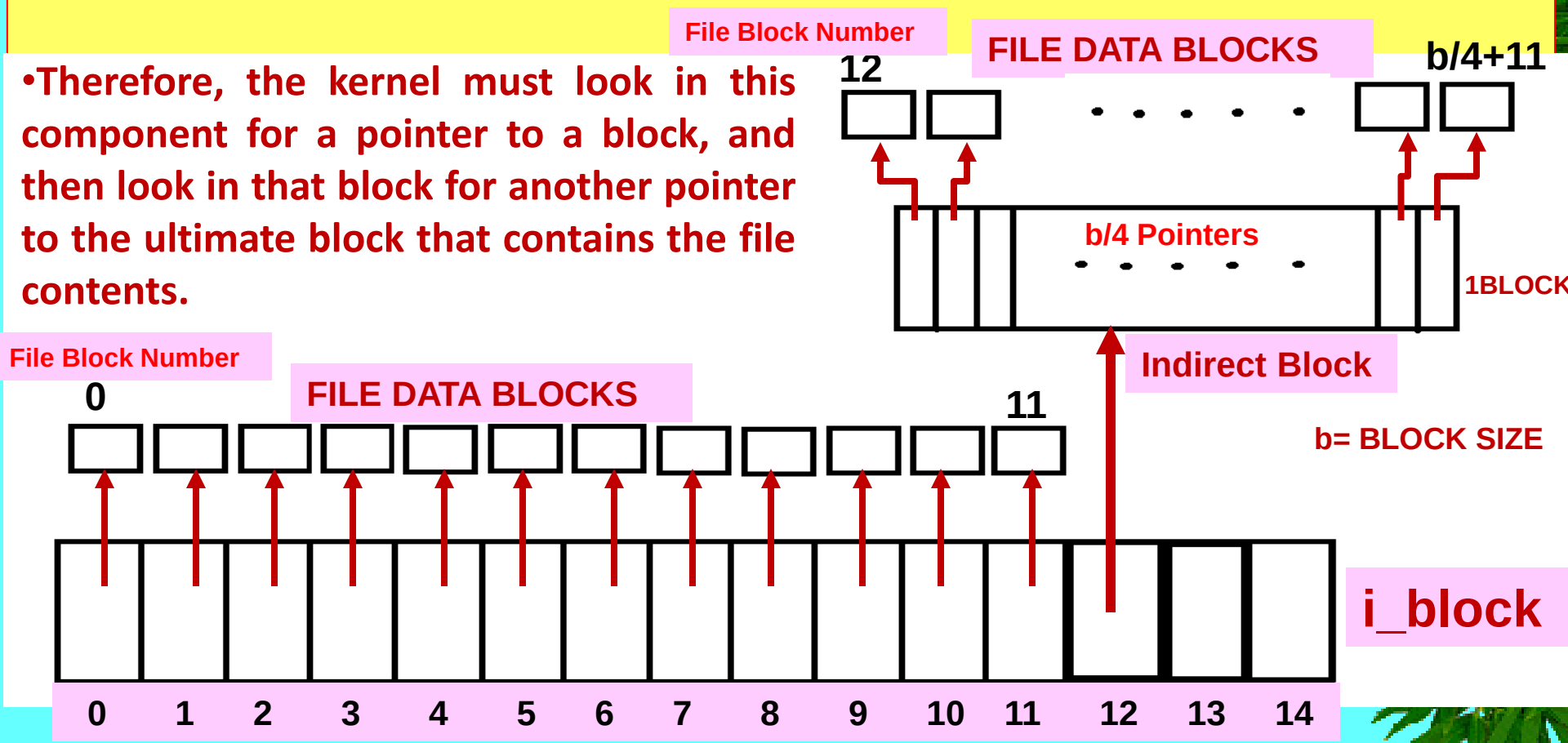
- The first 12 components ($i_block[0]$ to $i_block[11]$) yield the logical block numbers corresponding to the first 12 blocks of the file - to the blocks that have file block numbers from 0 to 11.
- If block size, $b=4\text{KB}$, then the first 12 components of i_block will point to 12 logical blocks each of size 4KB
- In otherwords, a file of maximum size = $12 \times 4\text{Kb} = 48\text{KB}$ can be stored or accessed using the first 12 components of the i_block array.



□ Data Blocks Addressing - 12th Component

- The component at index 12 contains the logical block number of a block, called **indirect block**, that represents a second-order array of logical block numbers.
- They correspond to the **file block numbers ranging from 12 to $b/4+11$** , where b is the filesystem's block size (each logical block number is stored in 4 bytes, so we divide by 4 in the formula).

• Therefore, the kernel must look in this component for a pointer to a block, and then look in that block for another pointer to the ultimate block that contains the file contents.

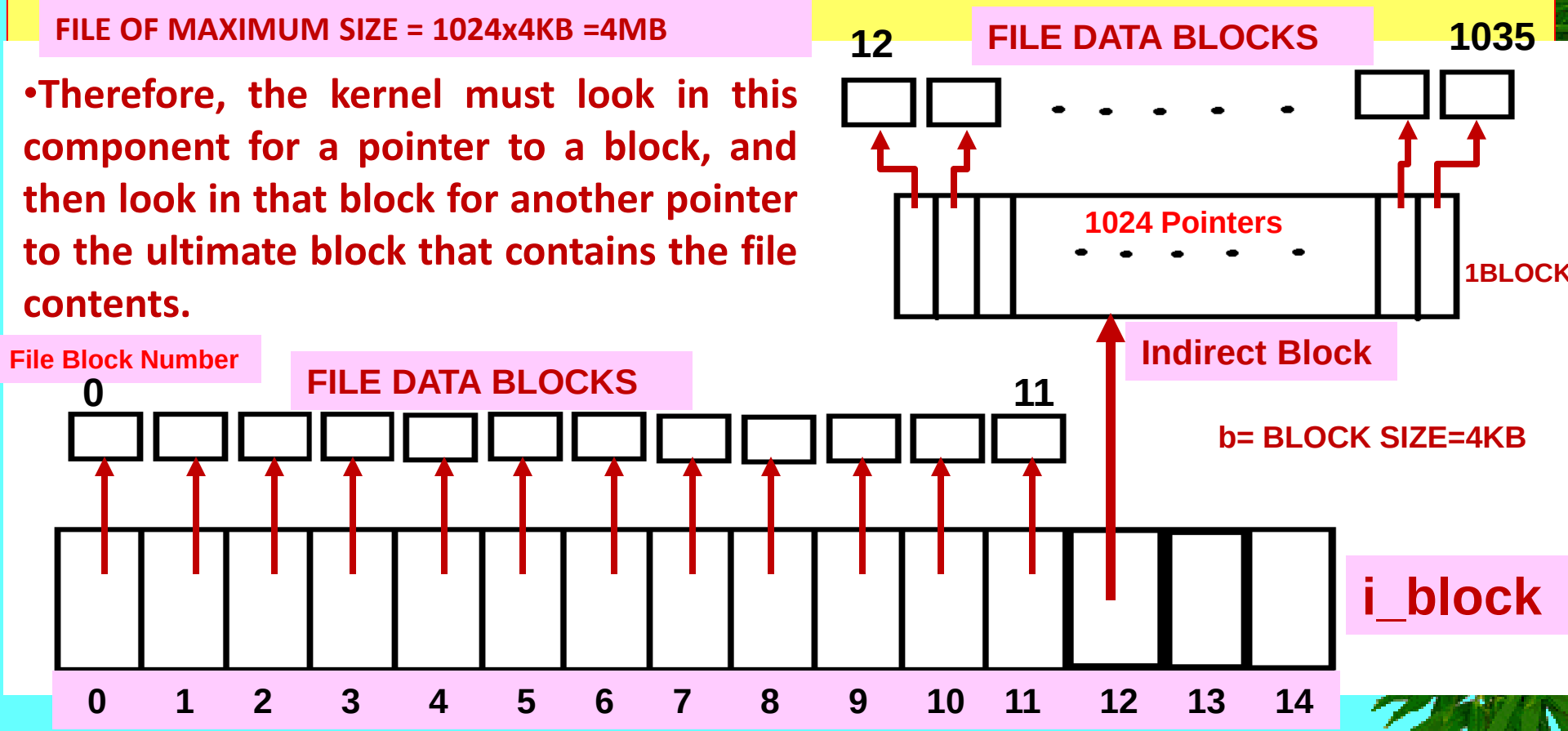


□ Data Blocks Addressing- 12th Component

- The component at index 12 contains the logical block number of a block, called **indirect block**, that represents a second-order array of logical block numbers.
- If filesystem's **block size b= 4KB = 4096 bytes**, Then the file block numbers ranging from 12 to $4096/4+11(1024+11 = 1035)$
- In otherwords, a file of maximum size = $1024 \times 4KB = 4MB$ can be stored or accessed using the 12 component of the i_block array.

FILE OF MAXIMUM SIZE = $1024 \times 4KB = 4MB$

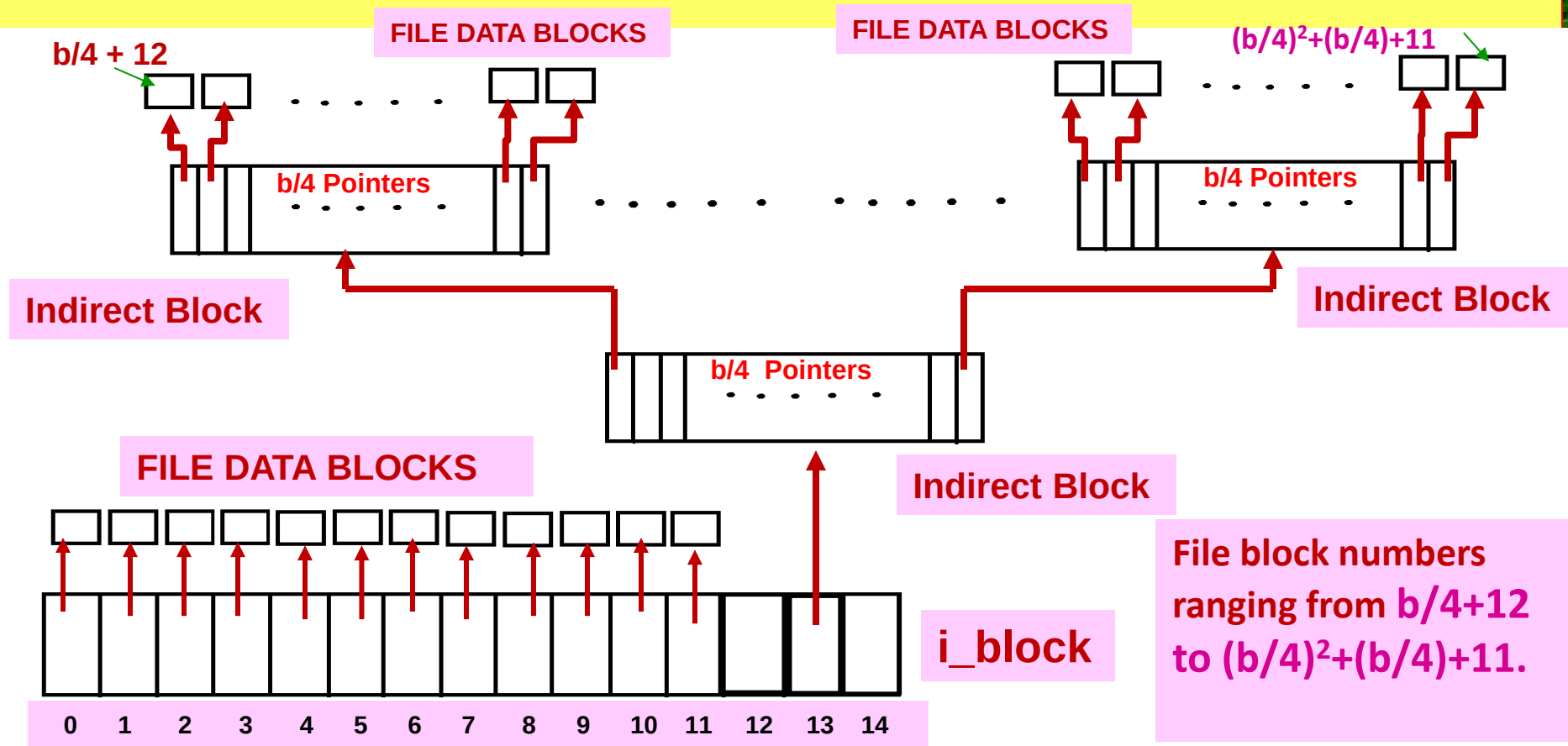
•Therefore, the kernel must look in this component for a pointer to a block, and then look in that block for another pointer to the ultimate block that contains the file contents.



Data Blocks Addressing

❑ Data Blocks Addressing - 13th Component

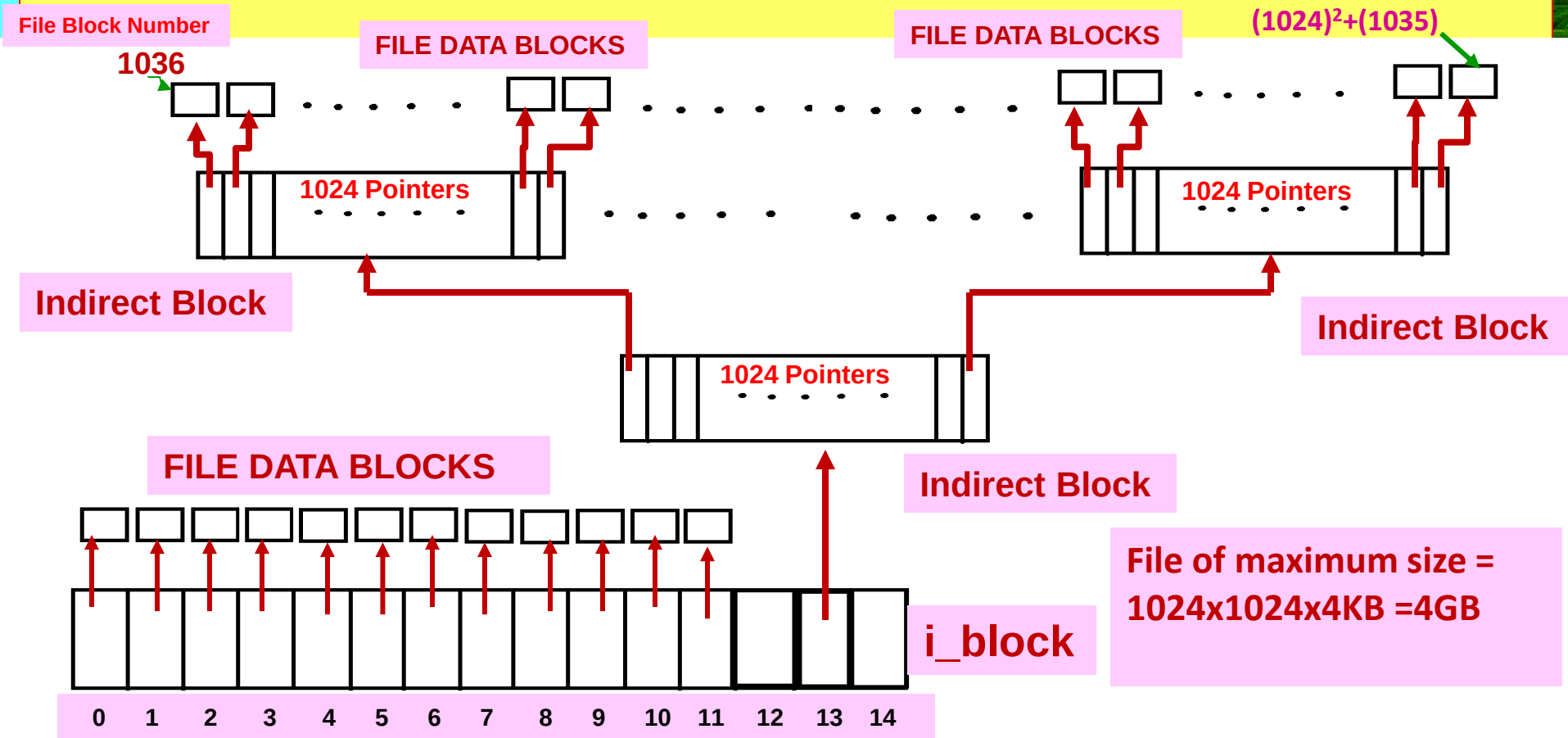
- The component at index 13 contains the logical block number of an indirect block containing a second-order array of logical block numbers; in turn, the entries of this second-order array point to third-order arrays, which store the logical block numbers that correspond to the file block numbers ranging from $b/4+12$ to $(b/4)^2+(b/4)+11$.



Data Blocks Addressing

❑ Data Blocks Addressing - 13th Component

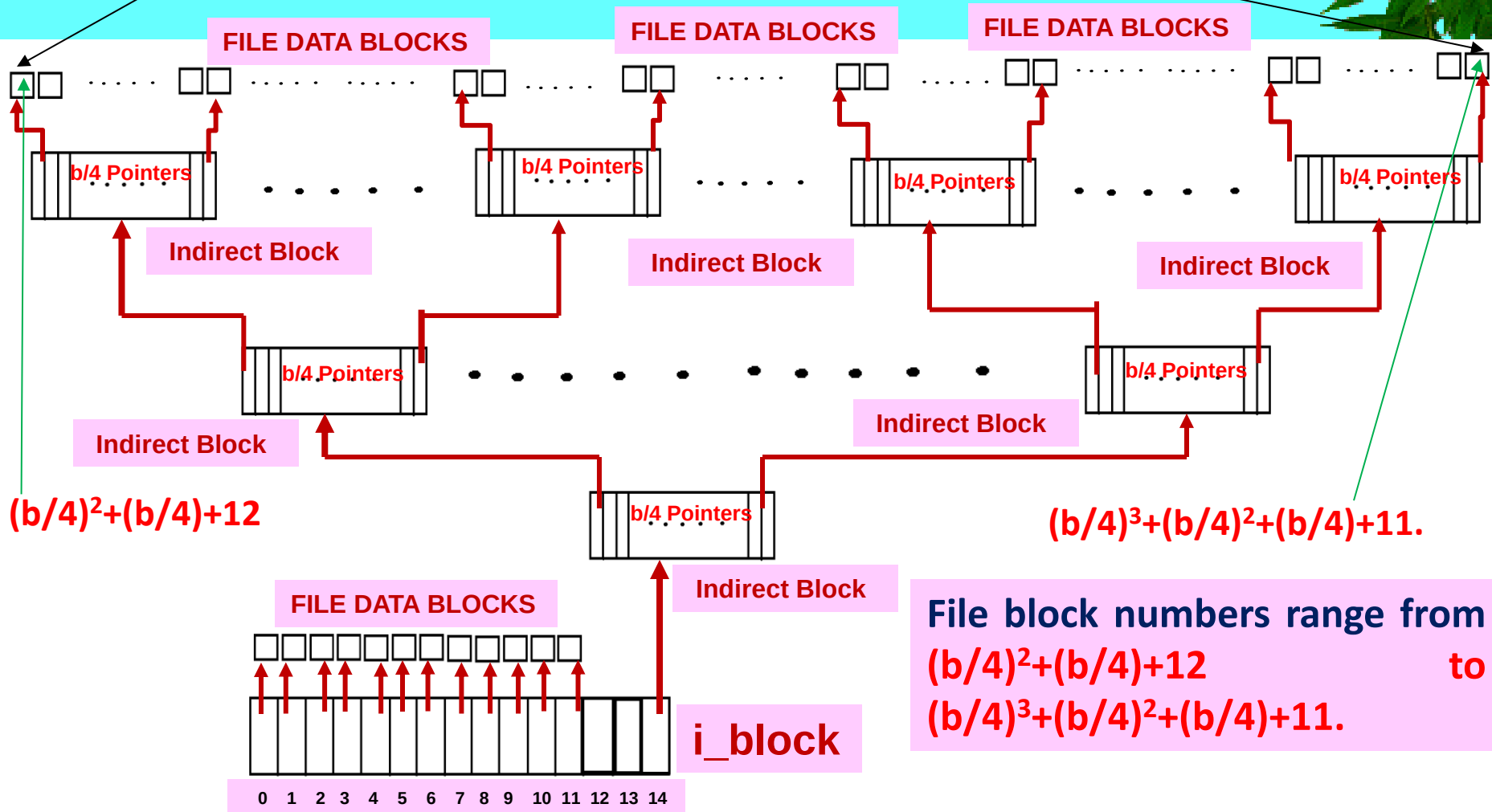
- The component at index 13 contains the logical block number of an indirect block containing a second-order array of logical block numbers; in turn, the entries of this second-order array point to third-order arrays, which store the logical block numbers that correspond to the file block numbers ranging from $1024+12=1036$ to $(1024)^2+(1024)+11= (1024)^2 + 1035$
- In otherwords, a file of maximum size = $1024 \times 1024 \times 4\text{KB} = 4\text{GB}$ can be stored or accessed using the 13 component of the `i_block` array.



□ Data Blocks Addressing - 14th Component

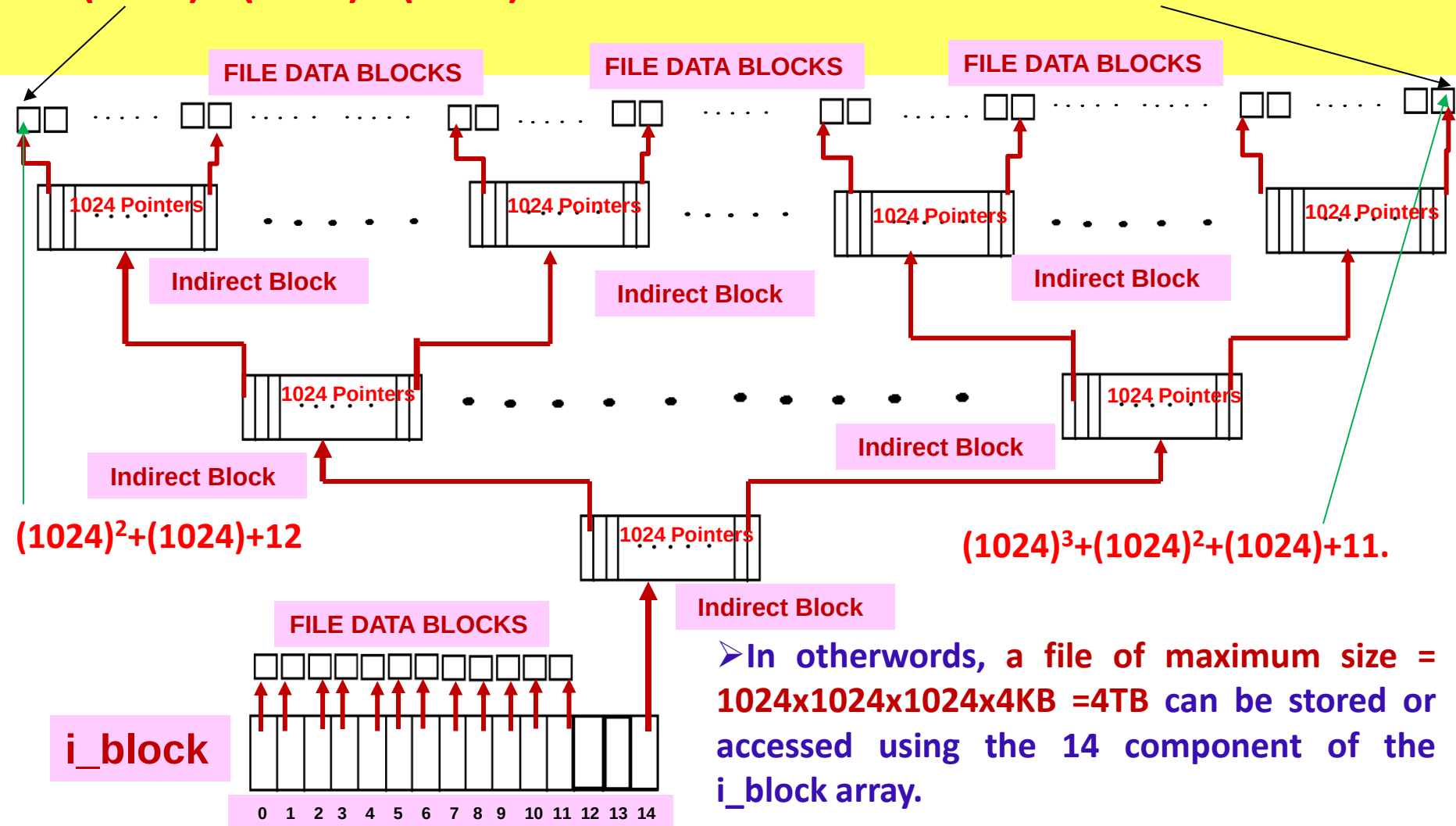
- Finally, the component at index 14 uses triple indirection: the fourth-order arrays store the logical block numbers corresponding to the file block numbers ranging from

$(b/4)^2+(b/4)+12$ to $(b/4)^3+(b/4)^2+(b/4)+11$.



□ Data Blocks Addressing - 14th Component

- If the blocksize, $b=4\text{KB}$, then the component at index 14 uses triple indirection: the fourth-order arrays store the logical block numbers corresponding to the file block numbers ranging from $(1024)^2+(1024)+12$ to $(1024)^3+(1024)^2+(1024)+11$.



□ Data Blocks Addressing

- As can be seen in the figure, the 15 components of the array are of 4 different types:
- The first 12 components yield the logical block numbers corresponding to the first 12 blocks of the file - to the blocks that have file block numbers from 0 to 11.
- The component at index 12 contains the logical block number of a block, called indirect block, that represents a second-order array of logical block numbers.
- They correspond to the file block numbers ranging from 12 to $b/4+11$, where b is the filesystem's block size (each logical block number is stored in 4 bytes, so we divide by 4 in the formula).
- Therefore, the kernel must look in this component for a pointer to a block, and then look in that block for another pointer to the ultimate block that contains the file contents.

Data Blocks Addressing

□ Data Blocks Addressing

- The component at index 13 contains the logical block number of an indirect block containing a second-order array of logical block numbers; in turn, the entries of this second-order array point to third-order arrays, which store the logical block numbers that correspond to the file block numbers ranging from $b/4+12$ to $(b/4)^2+(b/4)+11$.
- Finally, the component at index 14 uses triple indirection: the fourth-order arrays store the logical block numbers corresponding to the file block numbers ranging from $(b/4)^2+(b/4)+12$ to $(b/4)^3+(b/4)^2+(b/4)+11$.
- In Figure, the number inside a block represents the corresponding file block number. The arrows, which represent logical block numbers stored in array components, show how the kernel finds its way through indirect blocks to reach the block that contains the actual contents of the file.

❑ Data Blocks Addressing

- **For Small files :** If the file does not require more than 12 data blocks, every data can be retrieved in two disk accesses: one to read a component in the `i_block` array of the disk inode and the other to read the requested data block.
- **For larger files,** three or even four consecutive disk accesses may be needed to access the required block. In practice, this is a worst-case estimate, because dentry, inode, and page caches contribute significantly to reduce the number of real disk accesses.
- Notice also how the block size of the filesystem affects the addressing mechanism, because a larger block size allows the Ext2 to store more logical block numbers inside a single block.

❑ Data Blocks Addressing

- Table shows the upper limit placed on a file's size for each block size and each addressing mode.
- For instance, if the block size is 1,024 bytes and the file contains up to 268 kilobytes of data, the first 12 KB of a file can be accessed through direct mapping and the remaining 13-268 KB can be addressed through simple indirection.
- Files larger than 2 GB must be opened on 32-bit architectures by specifying the `O_LARGEFILE` opening flag.

Table - File-size upper limits for data block addressing

Block size	Direct	1-Indirect	2-Indirect	3-Indirect
1,024	12 KB	268 KB	64.26 MB	6.06 GB
2,048	24 KB	1.02 MB	513.02 MB	256.5 GB
4,096	48 KB	4.04 MB	4 GB	~ 4 TB

DISK FILE SYSTEM

□ Question:

- * Assume a data block size of 4KB, find the file block number for the given file offset 14320. Also specify the corresponding logical block number.

Given Block size = 4KB, File offset = 14320

File block number 0 → Data Block : bytes 0 to 4095

File block number 1 → Data Block : bytes 4096 to 8191

File block number 2 → Data Block : bytes 8192 to 12287

File block number 3 → Data Block : bytes 12288 to 16383

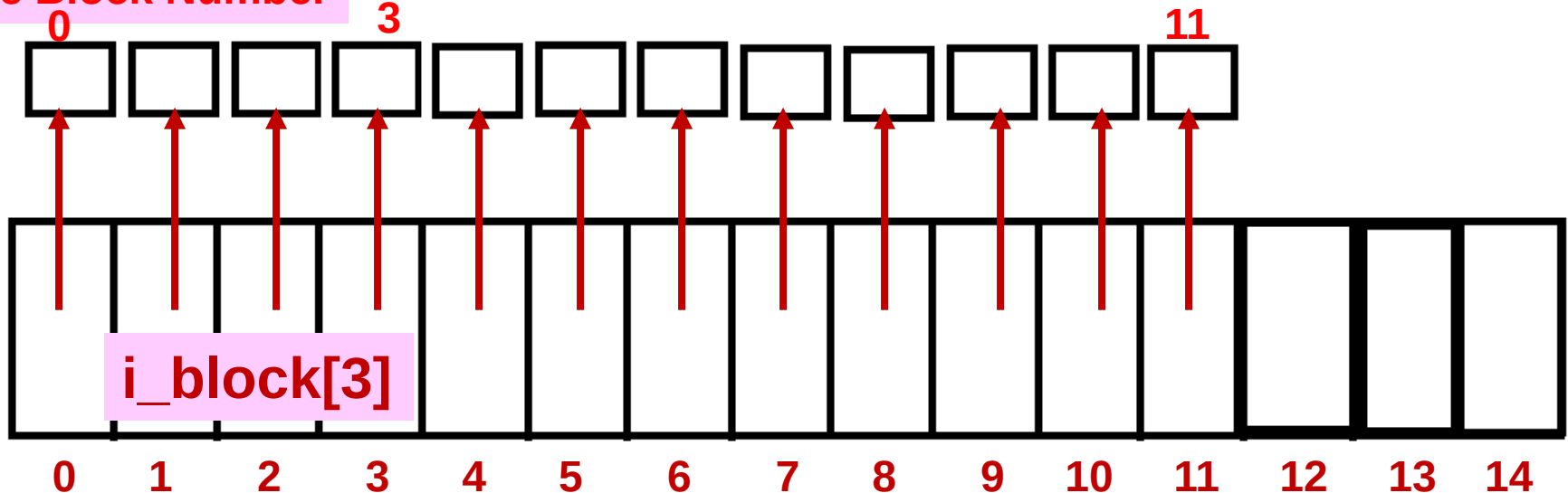
Hence, the file offset 14320 belongs to file block number 3

Also we can find the file block number by using the formula :

$$\text{File block number} = \text{File offset} / \text{Block size} = 14320 / 4096 = 3$$

For file block number 3, the corresponding logical block number is given by the `i_block[3]` component

File Block Number



DISK FILE SYSTEM

Question

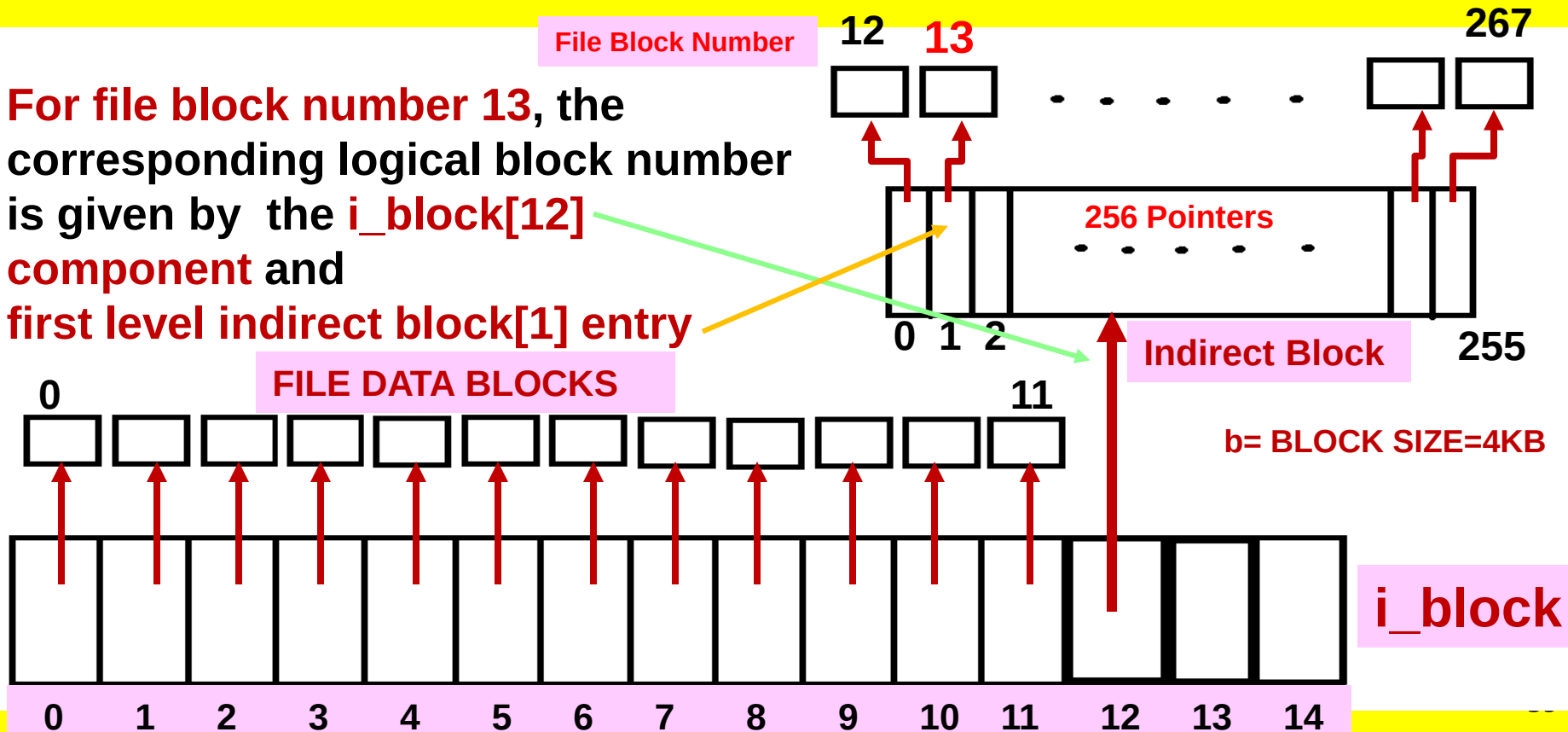
- Assume a data block size of 1KB, find the file block number for the given file offset 14320. Also specify the corresponding logical block number.

Given Block size = 1KB, File offset= 14320

we can find the file block number by using the formula :

$$\text{File block number} = \text{File offset} / \text{Block size} = 14320 / 1024 = 13$$

For file block number 13, the corresponding logical block number is given by the **i_block[12]** component and first level indirect block[1] entry



DISK FILE SYSTEM

□ Questions

- * Describe the two steps used to derive the logical block number of the corresponding data block from an offset f inside a file.
- * Assume a data block size of 4KB, find the file block number for the given file offset 14320. Also specify the corresponding logical block number.
- * Assume a data block size of 1KB, find the file block number for the given file offset 14320. Also specify the corresponding logical block number.
- * Assume a data block size of 4KB, find the file block number for the given file offset 348320. Also specify the corresponding logical block number.
- * Assume a data block size of 1KB, find the file block number for the given file offset 348320. Also specify the corresponding logical block number.
- * Assume a data block size of 4KB, find the file block number for the given file offset 6782341. Also specify the corresponding logical block number.
- * Explain with the help of a schematic, how Data Block Addressing is implemented in Ext2 filesystem.
- * Describe clearly how `i_block` array field in the disk inode is used to address the disk data blocks pertaining to a file.
- * Determine how many disk accesses are required to access a file data block in
 - a) one of the first 12 components `i_block[0]` to `[11]` of the `i_block` array.
 - b) 12th component `i_block[12]` of the `i_block` array.
 - c) 13th component `i_block[13]` of the `i_block` array
 - d) 14th component `i_block[14]` of the `i_block` array.

DISK FILE SYSTEM

□ **Questions**

- * With the help of a schematic, explain the following with respect to Data Block addressing considering a block size of 4KB:
 - (i) What do the first 12 components of the i_block array i.e., i_block[0] to i_block[11] contain?
What should be the maximum size of the file that can be addressed by these 12 components?
 - (ii) What does the 12th component of i_block array, i.e., i_block[12] contain? How many levels of indirect addressing is used?
How many data blocks pertaining to a file can be addressed by this component? Also specify the starting and ending numbers (indices) of these data block pointers. Explain with the help of a schematic.
 - (iii) What should be the maximum size of the file that can be addressed considering only 13th component of the array.
What should be the maximum size of the file that can be addressed by considering first 13 components of the i_block array?
 - (iv) What does the 14th component of i_block array, i.e., i_block[13] contain? How many levels of indirect addressing is used?
How many data blocks pertaining to a file can be addressed by this component Also specify the starting and ending numbers (indices) of these data block pointers. Explain with the help of a schematic.
What should be the maximum size of the file that can be addressed considering only 14th component of the array.
What should be the maximum size of the file that can be addressed by considering first 14 components of the i_block array?
 - (v) Determine the total amount of disk blocks required to store all the 15 components of the i_block array i.e., all direct block pointers and their all levels of indirect block pointers (second order array, third order array, fourth order array, etc).
 - (vi) Determine Maximum amount of disk blocks required to store all the block pointers and all file data blocks.

DISK FILE SYSTEM

□ Questions

- * With the help of a schematic, explain the following with respect to Data Block addressing considering a block size of 1 KB:
 - (i) What do the first 12 components of the `i_block` array i.e., `i_block[0]` to `i_block[11]` contain?
What should be the maximum size of the file that can be addressed by these 12 components?
 - (ii) What does the 12th component of `i_block` array, i.e., `i_block[12]` contain? How many levels of indirect addressing is used?
How many data blocks pertaining to a file can be addressed by this component? Also specify the starting and ending numbers (indices) of these data block pointers. Explain with the help of a schematic.
 - (iii) What should be the maximum size of the file that can be addressed considering only 13th component of the array.
What should be the maximum size of the file that can be addressed by considering first 13 components of the `i_block` array?
 - (iv) What does the 14th component of `i_block` array, i.e., `i_block[13]` contain? How many levels of indirect addressing is used?
How many data blocks pertaining to a file can be addressed by this component Also specify the starting and ending numbers (indices) of these data block pointers. Explain with the help of a schematic.
What should be the maximum size of the file that can be addressed considering only 14th component of the array.
What should be the maximum size of the file that can be addressed by considering first 14 components of the `i_block` array?
 - (v) Determine the total amount of disk blocks required to store all the 15 components of the `i_block` array i.e., all direct block pointers and their all levels of indirect block pointers (second order array, third order array, fourth order array, etc).
 - (vi) Determine Maximum amount of disk blocks required to store all the block pointers and all file data blocks.

DISK FILE SYSTEM

□ Questions

- * With the help of a schematic, explain the following with respect to Data Block addressing considering a block size of 2 KB:
 - (i) What do the first 12 components of the i_block array i.e., i_block[0] to i_block[11] contain? What should be the maximum size of the file that can be addressed by these 12 components?
 - (ii) What does the 12th component of i_block array, i.e., i_block[12] contain? How many levels of indirect addressing is used? How many data blocks pertaining to a file can be addressed by this component? Also specify the starting and ending numbers (indices) of these data block pointers. Explain with the help of a schematic.
 - (iii) What should be the maximum size of the file that can be addressed considering only 13th component of the array. What should be the maximum size of the file that can be addressed by considering first 13 components of the i_block array?
 - (iv) What does the 14th component of i_block array, i.e., i_block[13] contain? How many levels of indirect addressing is used? How many data blocks pertaining to a file can be addressed by this component Also specify the starting and ending numbers (indices) of these data block pointers. Explain with the help of a schematic. What should be the maximum size of the file that can be addressed considering only 14th component of the array. What should be the maximum size of the file that can be addressed by considering first 14 components of the i_block array?
 - (v) Determine the total amount of disk blocks required to store all the 15 components of the i_block array i.e., all direct block pointers and their all levels of indirect block pointers (second order array, third order array, fourth order array, etc).
 - (vi) Determine Maximum amount of disk blocks required to store all the block pointers and all file data blocks.

DISK FILE SYSTEM

□ Questions

- * With the help of a schematic, explain the following with respect to Data Block addressing considering a block size of 8 KB:
 - (i) What do the first 12 components of the `i_block` array i.e., `i_block[0]` to `i_block[11]` contain?
What should be the maximum size of the file that can be addressed by these 12 components?
 - (ii) What does the 12th component of `i_block` array, i.e., `i_block[12]` contain? How many levels of indirect addressing is used?
How many data blocks pertaining to a file can be addressed by this component? Also specify the starting and ending numbers (indices) of these data block pointers. Explain with the help of a schematic.
 - (iii) What should be the maximum size of the file that can be addressed considering only 13th component of the array.
What should be the maximum size of the file that can be addressed by considering first 13 components of the `i_block` array?
 - (iv) What does the 14th component of `i_block` array, i.e., `i_block[13]` contain? How many levels of indirect addressing is used?
How many data blocks pertaining to a file can be addressed by this component Also specify the starting and ending numbers (indices) of these data block pointers. Explain with the help of a schematic.
What should be the maximum size of the file that can be addressed considering only 14th component of the array.
What should be the maximum size of the file that can be addressed by considering first 14 components of the `i_block` array?
 - (v) Determine the total amount of disk blocks required to store all the 15 components of the `i_block` array i.e., all direct block pointers and their all levels of indirect block pointers (second order array, third order array, fourth order array, etc).
 - (vi) Determine Maximum amount of disk blocks required to store all the block pointers and all file data blocks.

DISK FILE SYSTEM

□ Questions

- * With the help of a schematic, explain the following with respect to Data Block addressing considering a block size of 64 KB:
 - (i) What do the first 12 components of the `i_block` array i.e., `i_block[0]` to `i_block[11]` contain?
What should be the maximum size of the file that can be addressed by these 12 components?
 - (ii) What does the 12th component of `i_block` array, i.e., `i_block[12]` contain? How many levels of indirect addressing is used?
How many data blocks pertaining to a file can be addressed by this component? Also specify the starting and ending numbers (indices) of these data block pointers. Explain with the help of a schematic.
 - (iii) What should be the maximum size of the file that can be addressed considering only 13th component of the array.
What should be the maximum size of the file that can be addressed by considering first 13 components of the `i_block` array?
 - (iv) What does the 14th component of `i_block` array, i.e., `i_block[13]` contain? How many levels of indirect addressing is used?
How many data blocks pertaining to a file can be addressed by this component Also specify the starting and ending numbers (indices) of these data block pointers. Explain with the help of a schematic.
What should be the maximum size of the file that can be addressed considering only 14th component of the array.
What should be the maximum size of the file that can be addressed by considering first 14 components of the `i_block` array?
 - (v) Determine the total amount of disk blocks required to store all the 15 components of the `i_block` array i.e., all direct block pointers and their all levels of indirect block pointers (second order array, third order array, fourth order array, etc).
 - (vi) Determine Maximum amount of disk blocks required to store all the block pointers and all file data blocks.

SOC 3010

OPERATING SYSTEM

FILE HOLES

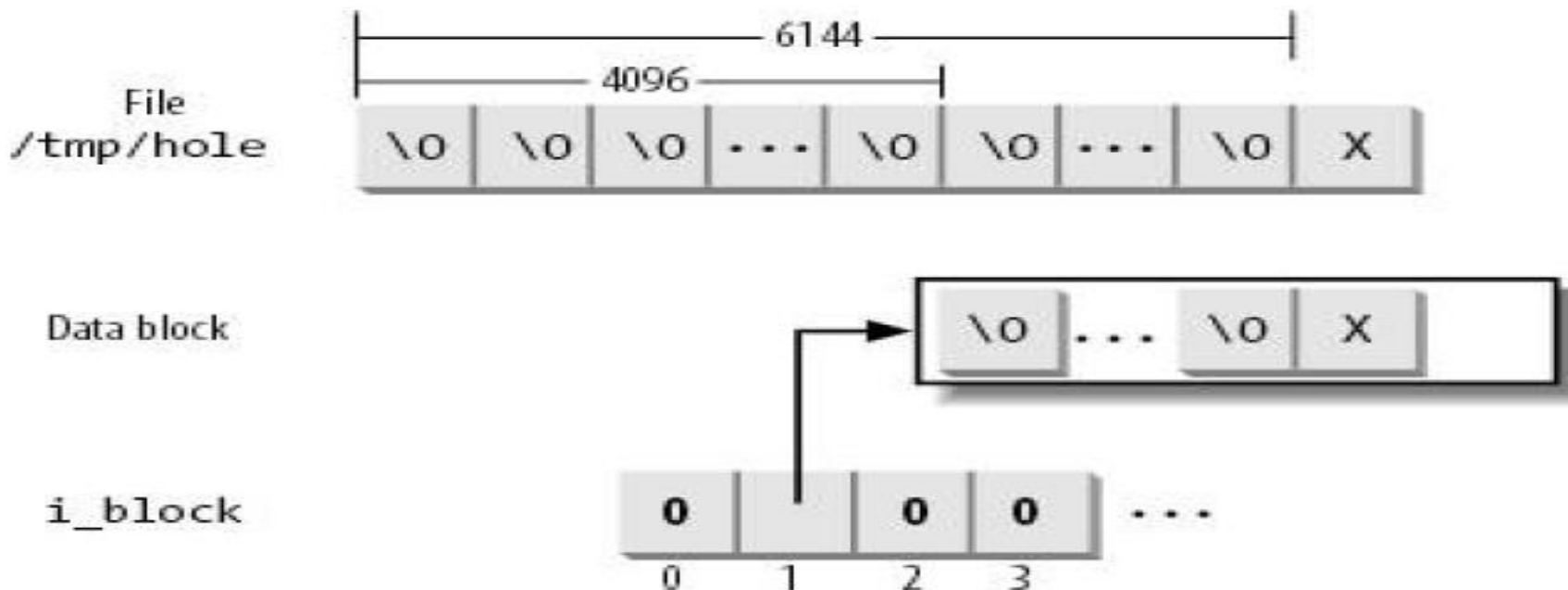
❑ File Holes

- A file hole is a portion of a regular file that contains null characters and is not stored in any data block on disk. Holes are a long-standing feature of Unix files.
- For instance, the following Unix command creates a file in which the first bytes are a hole:
\$ echo -n "X" | dd of=/tmp/hole bs=1024 seek=6
- Now /tmp/hole has 6,145 characters (6,144 null characters plus an X character), yet the file occupies just one data block on disk.
- File holes were introduced to avoid wasting disk space.
- They are used extensively by database applications and, more generally, by all applications that perform hashing on files.

❑ File Holes

- The Ext2 implementation of file holes is based on dynamic data block allocation: a block is actually assigned to a file only when the process needs to write data into it.
- The `i_size` field of each inode defines the size of the file as seen by the program, including the holes, i.e., `i_size = 6145` bytes
- While the `i_blocks` field stores the number of data blocks effectively assigned to the file (in units of 512 bytes). = 8 i.e, $8 \times 512 = 4\text{KB}$, that is nothing but 1 block (as `blocksize = 4KB`)

```
$ echo -n "X" | dd of=/tmp/hole bs=1024 seek=6
```



File Holes

❑ File Holes

- In the example of the `dd` command, suppose the `/tmp/hole` file was created on an Ext2 partition that has blocks of size 4,096.
- The `i_size` field of the corresponding disk inode stores the number 6,145, while the `i_blocks` field stores the number 8 (because each 4,096-byte block includes eight 512-byte blocks).
- The second element of the `i_block` array (corresponding to the block having file block number 1) stores the logical block number of the allocated block, while all other elements in the array are null.

Allocating a Data Block

❑ Allocating a Data Block

- When the kernel has to locate a block holding data for an Ext2 regular file, it invokes the `ext2_get_block( )` function. If the block does not exist, the function automatically allocates the block to the file.
- Remember that this function may be invoked every time the kernel issues a read or write operation on an Ext2 regular file; clearly, this function is invoked only if the affected block is not included in the page cache.
- The `ext2_get_block()` function handles the data structures and when necessary, invokes the `ext2_alloc_block( )` function to actually search for a free block in the Ext2 partition.
- If necessary, the function also allocates the blocks used for indirect addressing

❑ Releasing a Data Block

- When a process deletes a file or truncates it to 0 length, all its data blocks must be reclaimed.
- This is done by **ext2_truncate()**, which receives the address of the file's inode object as its parameter.
- The function essentially scans the disk inode's **i_block** array to locate all data blocks and all blocks used for the indirect addressing.
- These blocks are then released by repeatedly invoking **ext2_free_blocks()**.
- The **ext2_free_blocks()** function releases a group of one or more adjacent data blocks.
- Besides its use by **ext2_truncate()**, the function is invoked mainly when discarding the pre-allocated blocks of a file.
- Its parameters are:
 - inode** The address of the inode object that describes the file
 - block** The logical block number of the first block to be released
 - count** The number of adjacent blocks to be released

SOC 3010

OPERATING SYSTEM

JOURNALING FILE SYSTEMS

WISH YOU ALL THE BEST

❑ Journaling Filesystems

- As disks became larger, one design choice of traditional Unix filesystems (such as Ext2) turns out to be inappropriate.
- Updates to filesystem blocks might be kept in dynamic memory for long period of time before being flushed to disk.
- A dramatic event such as a power-down failure or a system crash might thus leave the filesystem in an inconsistent state.
- To overcome this problem, each traditional Unix filesystem is checked before being mounted; if it has not been properly unmounted, then a specific program executes an exhaustive, time-consuming check and fixes all the filesystem's data structures on disk.

Journaling Filesystems

❑ Journaling Filesystems

- For instance, the Ext2 filesystem status is stored in the **s_mount_state** field of the superblock on disk.
- The **e2fsck utility** program is invoked by the boot script to check the value stored in this field; if it is not equal to EXT2_VALID_FS, the filesystem was not properly unmounted, and therefore e2fsck starts checking all disk data structures of the filesystem.
- Clearly, the time spent checking the consistency of a filesystem depends mainly on the number of files and directories to be examined; therefore, it also depends on the disk size.

Journaling Filesystems

❑ Journaling Filesystems

- Nowadays, with filesystems reaching hundreds of gigabytes, a single consistency check may take hours.
- The involved downtime is unacceptable for every production environment or high-availability server.
- The goal of a journaling filesystem is to avoid running time-consuming consistency checks on the whole filesystem by looking instead in a special disk area that contains the most recent disk write operations named journal.
- Remounting a journaling filesystem after a system failure is a matter of a few seconds

Ext3 Journaling Filesystems

❑ Ext3 Journaling Filesystems

- The idea behind Ext3 journaling is to perform each high-level change to the filesystem in two steps.
 1. First, a copy of the blocks to be written is stored in the journal; when the I/O data transfer to the journal is completed (in short, data is committed to the journal), then the blocks are written in the filesystem.
 2. When the I/O data transfer to the filesystem terminates (data is committed to the filesystem), the copies of the blocks in the journal are discarded.

❑ Ext3 Journaling Filesystems

- While recovering after a system failure, the **e2fsck** program distinguishes the following two cases:
 1. **The system failure occurred before a commit to the journal** Either the copies of the blocks relative to the high-level change are missing from the journal or they are incomplete; in both cases, **e2fsck** ignores them.
 2. **The system failure occurred after a commit to the journal** The copies of the blocks are valid, and **e2fsck** writes them into the filesystem.
- In the first case, the high-level change to the filesystem is lost, but the filesystem state is still consistent.
- In the second case, **e2fsck** applies the whole high-level change, thus fixing every inconsistency due to unfinished I/O data transfers into the filesystem.

Ext3 Journaling Filesystems

❑ Ext3 Journaling Filesystems

- Journaling filesystem ensures consistency only at the system call level.
- For instance, a system failure that occurs while you are copying a large file by issuing several write() system calls will interrupt the copy operation, thus the duplicated file will be shorter than the original one.
- Furthermore, journaling filesystems do not usually copy all blocks into the journal.
- In fact, each filesystem consists of two kinds of blocks: those containing the so-called metadata and those containing regular data.

❑ Ext3 Journaling Filesystems

- In the case of Ext2 and Ext3, there are six kinds of metadata: superblocks, group block descriptors, inodes, blocks used for indirect addressing (indirection blocks), data bitmap blocks, and inode bitmap blocks.
- Other filesystems may use different metadata.
- Several journaling filesystems, such as SGI's XFS and IBM's JFS , limit themselves to logging the operations affecting metadata.
- In fact, metadata's log records are sufficient to restore the consistency of the on-disk filesystem data structures.
- However, since operations on blocks of file data are not logged, nothing prevents a system failure from corrupting the contents of the files.

Ext3 Journaling Filesystems

❑ Ext3 Journaling Filesystems

- The Ext3 filesystem, however, can be configured to log the operations affecting both the filesystem metadata and the data blocks of the files.
- Because logging every kind of write operation leads to a significant performance penalty, Ext3 lets the system administrator decide what has to be logged; in particular, it offers three different journaling modes:
 - **Journal**
 - **Ordered**
 - **Writeback**

Ext3 Journaling Filesystems

❑ Ext3 Journaling modes

❖ Journal

- All filesystem data and metadata changes are logged into the journal.
- This mode minimizes the chance of losing the updates made to each file, but it requires many additional disk accesses.
- For example, when a new file is created, all its data blocks must be duplicated as log records.
- This is the safest and slowest Ext3 journaling mode.

❑ Ext3 Journaling Filesystems

❖ Ordered

- Only changes to filesystem metadata are logged into the journal.
- However, the Ext3 filesystem groups metadata and relative data blocks so that data blocks are written to disk before the metadata.
- This way, the chance to have data corruption inside the files is reduced;
- for instance, each write access that enlarges a file is guaranteed to be fully protected by the journal.
- This is the default Ext3 journaling mode.

Ext3 Journaling Filesystems

❑ Ext3 Journaling Modes

❖ Writeback

- ★ Only changes to filesystem metadata are logged; this is the method found on the other journaling filesystems and is the fastest mode.
- ❑ The journaling mode of the Ext3 filesystem is specified by an option of the mount system command.
- ★ For instance, to mount an Ext3 filesystem stored in the /dev/sda2 partition on the /jdisk mount point with the "writeback" mode, the system administrator can type the command:
mount -t ext3 -o data=writeback /dev/sda2 /jdisk
- ★ The Ext3 journal is usually stored in a hidden file named **.journal** located in the root directory of the filesystem.

DISK FILE SYSTEM

□ Questions

- * State the purpose of **sparse_super** feature flag in Ext4 file system.

- * If disk block size = 1KB, # of inodes = 8192 inodes and 8K Blockgroups then determine the following
 - a) size of superblock,
 - b) size of blockgroup descriptors,
 - c) size of data block bitmap,
 - d) size of inode bitmap,
 - e) size of inode and size of inode table,
 - f) number of data blocks and size of data blocks
 - g) Size of one blockgroup
 - h) Size of the disk partition.

- * If disk block size = 2 KB, # of inodes = 8192 inodes and 16K Blockgroups then determine the following
 - a) size of superblock,
 - b) size of blockgroup descriptors,
 - c) size of data block bitmap,
 - d) size of inode bitmap,
 - e) size of inode and size of inode table,
 - f) number of data blocks and size of data blocks
 - g) Size of one blockgroup
 - h) Size of the disk partition.

DISK FILE SYSTEM

Questions

- * Specify the File size upper limits for data block addressing in the following table :

Block Size	Direct	1-Indirect	2-indirect	3-indirect
1KB				
2KB				
4KB				
8KB				
16KB				

- * Explain what is meant by a file hole. Give an example.
- * Write the file size and total number of blocks assigned to a file **hole** when the following commands are executed. Also show the `i_block` array and file `/tmp/hole` data block contents.

```
$ echo -n "X" | dd of=/tmp/hole bs=1024 seek=6  
$ echo -n "X" | dd of=/tmp/hole bs=4096 seek=6  
$ echo "HELLO" | dd of=/tmp/hole bs=1024 seek=10  
$ echo "HELLO" | dd of=/tmp/hole bs=4096 seek=10
```
- * State the purpose of the following commands :

```
$ mke2fs -t ext4 -b 4096 /dev/sda1  
$fsck
```
- * State the purpose of the following commands :

```
$ mount -t ext4 /dev/fd0 /floppy  
$mount -t ext4 -o data=writeback /dev/sda2 /jdisk
```
- * State the purpose of the following commands :

```
$setfacl -m "u:user_name:rwX" file1  
$setfacl -m "g:group_name:rwX" file1  
$getfacl file1
```

DISK FILE SYSTEM

❑ **Questions**

- * Write a linux command to display only the contents of the superblock of ext4 filesystem stored on the disk partition /dev/sda1.
- * Write a linux command to display only the contents of the block group descriptors of ext4 filesystem stored on the disk partition /dev/sda1.
- * State what happens when the following linux commands are executed. Clearly write the outputs obtained.
 - \$ dumpe2fs -h /dev/sda1**
 - \$ dumpe2fs -g /dev/sda1**
- * State what happens when the following linux commands are executed. Clearly write the outputs obtained.
 - \$ tune2fs -l /dev/sda1**
 - \$ tune2fs -l /dev/sda1 | grep "Default mount options"**
 - \$ tune2fs -o acl /dev/sda1**
- * State what happens when the following linux commands are executed. Clearly write the outputs obtained.
 - \$ lsblk -O**
 - \$ fdisk -l**
- * State what happens when the following linux commands are executed. Clearly write the outputs obtained.
 - \$ ls -la**
 - \$ lscpu**
 - \$ lscpc**
- * Define Journaling and explain why is it required.
- * Explain the idea behind Ext3/4 Journaling to perform each high-level change to the filesystem.

DISK FILE SYSTEM

□ Questions

- * Describe how journaling is used to recover from system crashes.**
- * Explain the two kinds of blocks in filesystem that need to be logged to the Journal.**
- * How does the kernel check whether the filesystem was unmounted properly previously or not?**
- * State the purpose of `s_mount_state` field of the superblock on disk.**
- * Explain the three different Journaling modes provided by Ext3/Ext4. State the merits and demerits of each of these modes.**
- * Explain the following Journaling modes indicating clearly their advantages and disadvantages.**
 - a) Journal**
 - b) ordered**
 - c) writeback**
- * Explain how Journaling mode of the Ext3/Ext4 filesystem can be specified at the time of mounting the filesystem.**
- * Write the linux command to mount an Ext3 filesystem stored in the `/dev/sda2` partition on the `/jdisk` mount point with the "writeback" mode.**
- * The Ext3 journal is usually stored in a hidden file namedlocated in the root directory of the filesystem.**

SOC 3010 OPERATING SYSTEMS

Reading Assignments

Chapter 18 of Text Book

Text Book

➤ Understanding the Linux Kernel

- Daniel P/ Bovet and Marco Cesati
Edition, O' Reilly Media 2005, ISBN 978-0596005658**

SOC 3010
OPERATING SYSTEM

END OF
WEEK 11 LECTURE 1

FILE : ON DISK FILE
SYSTEM

WISH YOU ALL THE BEST

INHA UNIVERSITY TASHKENT

FALL SEMESTER 2024

SOC 3010

OPERATING SYSTEM

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR

DR. A. R. NASEER

HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

